Max-Flow Min-Cut*

Daniel Mortensen and Florance Lapierre

April 14, 2026

Contents

1 Introduction and Example **2**
1.1 Definitions . . . 2

2 Min-Cut/Max-Flow **2**
2.1 Cuts Bound Flows . . . 3
2.2 Are they Equal? . . . 3
2.2.1 Matchings . . . 3
2.2.2 Alternating and Augmenting Paths . . . 4
2.2.3 Beyond Bipartite . . . 5
2.3 Hall's Theorem . . . 6
2.4 Every Cut Is a Bipartite Interface . . . 6

3 The Ford–Fulkerson Algorithm **7**
3.1 From Bipartite Matching to Network Flow . . . 7
3.2 Residual Graphs and Augmenting Paths . . . 7
3.3 Ford–Fulkerson on the Bipartite Network . . . 7
3.4 Max-Flow Algorithm . . . 9
3.5 Example: The General Network . . . 9

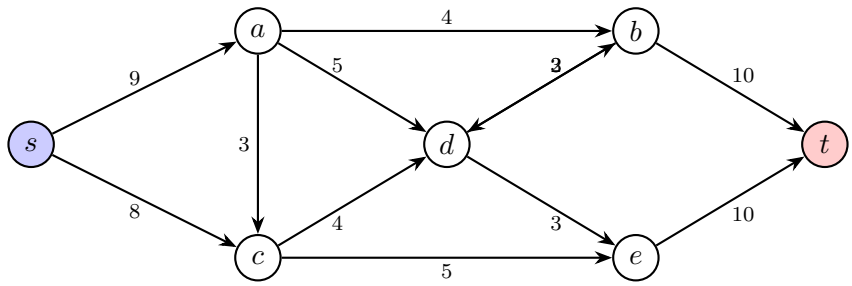
4 The Max-Flow Min-Cut Theorem **12**
4.1 Statement . . . 12
4.2 Proof . . . 12

5 Related Results **12**
5.1 Menger's Theorem . . . 13
5.2 König–Egerváry Theorem . . . 13
5.3 Birkhoff–von Neumann Theorem . . . 13

*can't take credit, but it's sure catchy

1 Introduction and Example

Many problems reduce to pushing as much material as possible through a network of limited-capacity links. Consider the following directed graph, where each edge has a maximum capacity:



We want to send flow from the **source** s to the **sink** t. Each edge can carry flow up to its capacity, and at every intermediate node, flow in must equal flow out. The network has a rich structure: the node d sits at the centre, the edges $b \to d$ and $d \to b$ form a cycle, and the cross-connections $a \to c$, $a \to d$, $c \to d$ create multiple overlapping routes from s to t. Different paths have different bottlenecks.

The central question is: *what is the maximum total flow from s to t, and where is the bottleneck?* We will develop the tools to answer this in Sections 2–4, then return to this network for a complete worked example in Section 4.

1.1 Definitions

Definition 1 (Network)**.** *A **network** is a directed graph $G = (V, E)$ without multiple arcs that contains a non-negative capacity function $c : E \to \mathbb{R}_{\geq 0}$.*

Definition 2 (Flow network)**.** *When a **network** contains a **source**, denoted s, and a **sink**, denoted t [1], then (G, c, s, t) is said to be a **flow network**.*

Definition 3 (Flow)**.** *A **flow** is a function $f : E \to \mathbb{R}_{\geq 0}$ satisfying:*

1. ***Capacity**: $0 \leq f(u, v) \leq c(u, v)$ for all $(u, v) \in E$.*
2. ***Conservation**: for every $v \in V \setminus \{s, t\}$,*

$$\sum_{u:(u,v)\in E} f(u, v) = \sum_{w:(v,w)\in E} f(v, w).$$

Definition 4 (s-t cut)**.** *An s-t **cut** (S, T) is a partition $V = S \cup T$ with $s \in S$ and $t \in T$. Its **capacity** is*

$$c(S, T) = \sum_{\substack{u\in S,\, v\in T \\ (u,v)\in E}} c(u, v).$$

2 Min-Cut/Max-Flow

The goal is to show that the maximum flow equals the minimum cut. The argument has three parts:

[1] ...because t totally makes you think sink...

1. **Cuts bound flows** (Section 2.1)—we revisit why every cut is a bipartite bottleneck in general graphs.
2. **Ford–Fulkerson algorithm** (Section 3)—we construct an algorithm that repeatedly pushes flow along augmenting paths until none remain.
3. **The theorem** (Section 4)—when the algorithm terminates, the final residual graph reveals a cut whose capacity equals the flow value, proving $\max |f| = \min c(S,T)$.

2.1 Cuts Bound Flows

Claim. For any flow f and any s-t cut (S,T), $|f| \leq c(S,T)$.

Proof. Let (S,T) be any s-t cut and f any flow.

Step 1. Every unit of flow from s to t must cross the cut at least once, because $s \in S$ and $t \in T$. Some flow may also cross back from T to S. Therefore the value of the flow equals the *net* flow across the cut:

$$|f| = \underbrace{\sum_{\substack{u \in S \\ v \in T}} f(u,v)}_{\text{flow from } S \text{ to } T} - \underbrace{\sum_{\substack{u \in T \\ v \in S}} f(u,v)}_{\text{flow from } T \text{ to } S} .$$

Step 2. The second sum is non-negative (all flows are ≥ 0), so dropping it can only increase the right-hand side:

$$|f| \leq \sum_{\substack{u \in S \\ v \in T}} f(u,v).$$

Step 3. Each term $f(u,v)$ satisfies the capacity constraint $f(u,v) \leq c(u,v)$, so:

$$\sum_{\substack{u \in S \\ v \in T}} f(u,v) \leq \sum_{\substack{u \in S \\ v \in T}} c(u,v) = c(S,T).$$

and hence, $|f| \leq c(S,T)$. □

The key insight is knowing that finding a flow f and a cut (S,T) with $|f| = c(S,T)$ actually guarantees that f is the maximum flow, and (S,T) is a minimum cut. The max-flow min-cut theorem (Section 5) guarantees that such a pair always exists.

2.2 Are they Equal?

2.2.1 Matchings

A **matching** $M \subseteq E$ is a set of edges with no shared endpoints. A vertex is **matched** if it belongs to some edge in M; otherwise it is **free**. A **maximum matching** maximises $|M|$. A **perfect matching** matches every vertex ($|M| = |U| = |V|$).

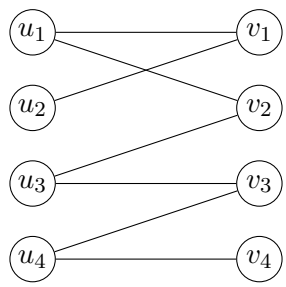
2.2.2 Alternating and Augmenting Paths

Given a matching M, an **alternating path** alternates between edges in M and edges not in M. An **augmenting path** is an alternating path whose two endpoints are both free.

If an augmenting path exists, flipping every edge along it (matched $\leftrightarrow$ unmatched) yields a matching of size $|M| + 1$.

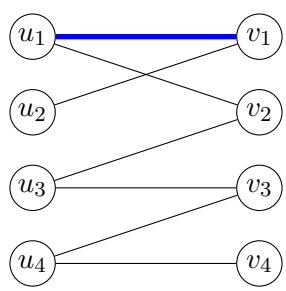
Theorem 1 (Berge, 1957)**.** *A matching M is maximum if and only if there is no augmenting path with respect to M.*

Consider the bipartite graph with $U = \{u_1, u_2, u_3, u_4\}$, $V = \{v_1, v_2, v_3, v_4\}$ and edges:



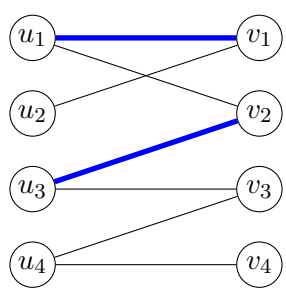
Note that u_2 has only one neighbour (v_1), so any perfect matching must assign $u_2 \to v_1$. The algorithm does not know this in advance.

Step 1. $M = \emptyset$. Search from u_1 (free): find v_1 (free). Augmenting path u_1–v_1. Augment.



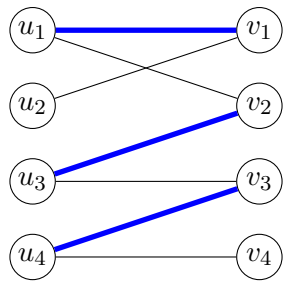
$M = \{(u_1, v_1)\}$, $|M| = 1$.

Step 2. Search from u_3 (free): find v_2 (free). Augmenting path u_3–v_2. Augment.



$M = \{(u_1, v_1), (u_3, v_2)\}$, $|M| = 2$.

Step 3. Search from u_4 (free): find v_3 (free). Augmenting path u_4–v_3. Augment.

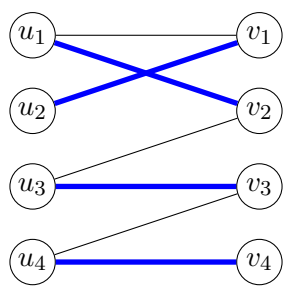


$M = \{(u_1, v_1), (u_3, v_2), (u_4, v_3)\}$, $|M| = 3$.

Step 4. The only free vertex in U is u_2. Its sole neighbour is v_1, which is matched to u_1. No free vertex is directly reachable, so BFS must follow alternating edges through the matching:

$$\underset{\text{free}}{u_2} \xrightarrow{\text{unmatched}} v_1 \xrightarrow{\text{matched}} u_1 \xrightarrow{\text{unmatched}} v_2 \xrightarrow{\text{matched}} u_3 \xrightarrow{\text{unmatched}} v_3 \xrightarrow{\text{matched}} u_4 \xrightarrow{\text{unmatched}} \underset{\text{free}}{v_4}$$

This is an augmenting path of length 7 (four unmatched edges, three matched edges). Flipping every edge along the path yields:



$M = \{(u_2, v_1), (u_1, v_2), (u_3, v_3), (u_4, v_4)\}$, $|M| = 4$. This is a perfect matching.

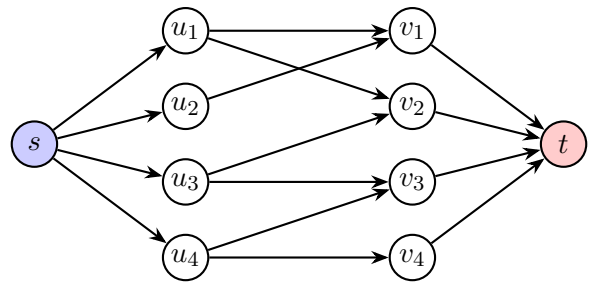
Step 5. No free vertices remain. No augmenting path exists. By Berge's theorem, M is a maximum matching.

The key observation is Step 4: the algorithm rerouted three existing matched edges through a single augmenting path. The "cascade" $v_1 \to u_1 \to v_2 \to u_3 \to v_3 \to u_4$ displaced every previous assignment to make room for u_2. Without this ability to undo prior choices, a greedy algorithm would be stuck at $|M| = 3$.

2.2.3 Beyond Bipartite

Flow and cuts apply not only to bipartite networks but to arbitrary directed graphs. The reason is that every cut in a general graph *already is* a bipartite bottleneck.

To illustrate, consider the following flow network built from the bipartite graph in Section 1 by adding a source s and sink t with unit-capacity edges:



2.3 Hall's Theorem

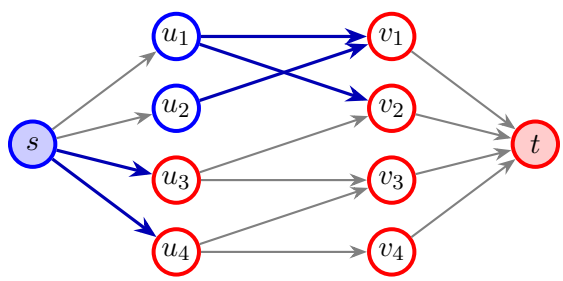
Theorem 2 (Hall, 1935)**.** *A bipartite graph $G = (U \cup V, E)$ has a matching saturating every vertex in U iff $|N(S)| \geq |S|$ for all $S \subseteq U$.*

Hall's condition is equivalent to every s-t cut having capacity at least $|U|$ in the corresponding flow network.

2.4 Every Cut Is a Bipartite Interface

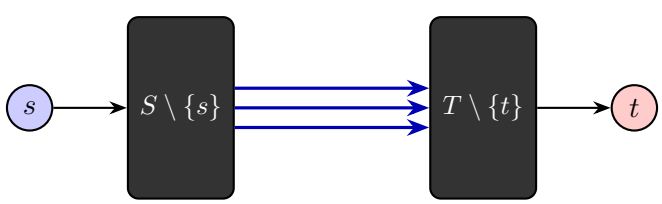
Take any s-t cut (S, T) in a directed graph. The edges crossing from S to T form a bipartite structure: each has one endpoint in S and one in T. Every unit of flow from s to t must cross this interface.

Consider the cut $S = \{s, u_1, u_2\}$, $T = \{u_3, u_4, v_1, v_2, v_3, v_4, t\}$ in our flow network:



The five bold edges cross from $S = \{s, u_1, u_2\}$ to $T = \{u_3, u_4, v_1, v_2, v_3, v_4, t\}$ (capacity 5); the gray edges are internal plumbing. No matter what happens inside S or T, flow from s to t cannot exceed 5 through this cut.

From the perspective of the cut, the internal structure on each side is irrelevant—only the crossing matters:



bipartite crossing: capacity = 5

The black boxes hide arbitrarily complex subgraphs. All that determines the throughput bound is the capacity of the edges between them.

3 The Ford–Fulkerson Algorithm

3.1 From Bipartite Matching to Network Flow

A bipartite matching problem is a network flow problem in disguise. We convert the bipartite graph from Section 1 into a flow network by adding a source s with capacity-1 edges to every $u \in U$, a sink t with capacity-1 edges from every $v \in V$, and directing the original edges from U to V with capacity 1. All capacities are 1. (This is the same network introduced in Section 3.)

An integer maximum flow in this network corresponds exactly to a maximum matching: each unit of flow along $s \to u_i \to v_j \to t$ represents the matched edge (u_i, v_j).

3.2 Residual Graphs and Augmenting Paths

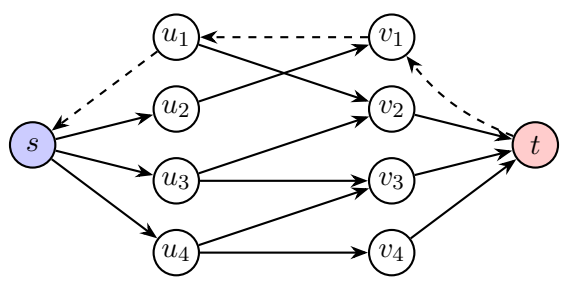
Given flow f, the **residual graph** $G_f = (V, E_f)$ contains a **forward edge** (u, v) with capacity $c_f(u, v) = c(u, v) - f(u, v)$ whenever this is positive, and a **backward edge** (v, u) with capacity $c_f(v, u) = f(u, v)$ whenever $f(u, v) > 0$. An **augmenting path** is any $s \to t$ path in G_f. Backward edges allow the algorithm to undo previously pushed flow—the same mechanism as augmenting paths in matching.

3.3 Ford–Fulkerson on the Bipartite Network

We now trace the Ford–Fulkerson algorithm on the bipartite flow network. At each step we show the residual graph, find an augmenting $s \to t$ path, and push one unit of flow. Edges labelled with their residual capacity; forward edges are solid, backward edges are dashed.

Iteration 1. All flows are zero. The residual graph equals the original network. BFS from s finds the path $s \to u_1 \to v_1 \to t$ with bottleneck $\delta = 1$. Push 1 unit.

Flow: $f(s, u_1) = f(u_1, v_1) = f(v_1, t) = 1$. Total $|f| = 1$.



Note the backward edges $u_1 \dashleftarrow v_1$, $s \dashleftarrow u_1$, and $v_1 \dashleftarrow t$: these allow future iterations to undo this flow.

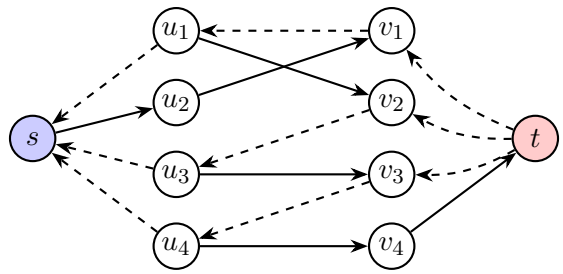
Iteration 2. BFS from s finds $s \to u_3 \to v_2 \to t$. Push 1 unit.

Flow: additionally $f(s, u_3) = f(u_3, v_2) = f(v_2, t) = 1$. Total $|f| = 2$.

Iteration 3. BFS from s finds $s \to u_4 \to v_3 \to t$. Push 1 unit.

Flow: additionally $f(s, u_4) = f(u_4, v_3) = f(v_3, t) = 1$. Total $|f| = 3$.

The residual graph after three iterations:



Iteration 4. BFS from s. The only forward edge from s is $s \to u_2$. From u_2, the only forward edge is $u_2 \to v_1$. But $v_1 \to t$ is saturated (only the backward edge $t \to v_1$ exists). So BFS must use the backward edge $v_1 \to u_1$ (undoing the flow $u_1 \to v_1$), then the forward edge $u_1 \to v_2$, then backward $v_2 \to u_3$, then forward $u_3 \to v_3$, then backward $v_3 \to u_4$, then forward $u_4 \to v_4$, then forward $v_4 \to t$.

The augmenting path in the residual graph:

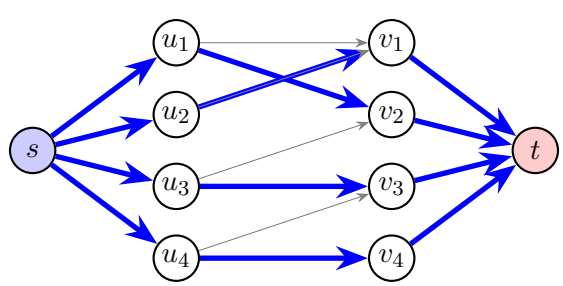
$$s \to u_2 \to v_1 \dashrightarrow u_1 \to v_2 \dashrightarrow u_3 \to v_3 \dashrightarrow u_4 \to v_4 \to t$$

where $\to$ denotes a forward edge and $\dashrightarrow$ a backward edge (undoing existing flow).

Bottleneck $\delta = 1$ (all residual capacities are 1). Push 1 unit along this path. For each forward edge, increase flow by 1; for each backward edge, *decrease* flow by 1 on the reverse arc:

Edge in path	**Type**	**Update**
$s \to u_2$	forward	$f(s, u_2) : 0 \to 1$
$u_2 \to v_1$	forward	$f(u_2, v_1) : 0 \to 1$
$v_1 \to u_1$	backward	$f(u_1, v_1) : 1 \to 0$
$u_1 \to v_2$	forward	$f(u_1, v_2) : 0 \to 1$
$v_2 \to u_3$	backward	$f(u_3, v_2) : 1 \to 0$
$u_3 \to v_3$	forward	$f(u_3, v_3) : 0 \to 1$
$v_3 \to u_4$	backward	$f(u_4, v_3) : 1 \to 0$
$u_4 \to v_4$	forward	$f(u_4, v_4) : 0 \to 1$
$v_4 \to t$	forward	$f(v_4, t) : 0 \to 1$

Total $|f| = 4$. The resulting flow (and corresponding matching):



Iteration 5. In the residual graph, all edges from s are saturated (backward only). No $s \to t$ path exists. The algorithm terminates.

The set of vertices reachable from s in the final residual graph is $S^* = \{s\}$ (no forward edge leaves s). The minimum cut is $(\{s\}, V \setminus \{s\})$ with capacity 4, confirming $\max |f| = \min c(S,T) = 4$.

Comparison with Section 1. The flow in iteration 4 corresponds exactly to the augmenting path in Step 4 of the matching algorithm. The backward edges in the residual graph play the same role as the matched edges in the alternating path: they let the algorithm *undo* previous flow assignments. Ford–Fulkerson on the flow network is the augmenting-paths algorithm in disguise.

This connection runs deeper: the minimum cut in the flow network corresponds to a *minimum vertex cover* in the original bipartite graph—a smallest set of vertices that touches every edge. The fact that max matching always equals min vertex cover in bipartite graphs is König's theorem (Section 5.2), and it is exactly what the max-flow min-cut theorem becomes when applied to bipartite matching.

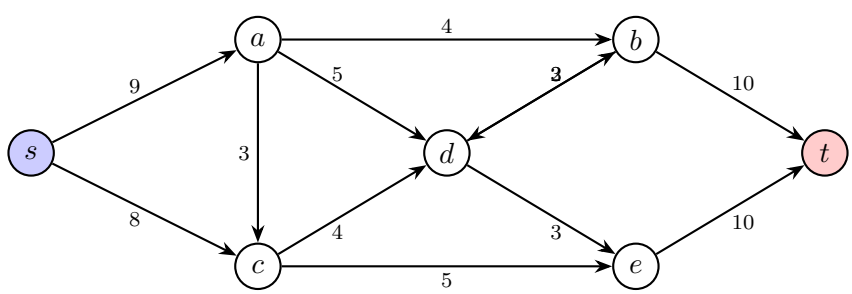
3.4 Max-Flow Algorithm

Input: Flow network (G, c, s, t).
Output: Maximum flow f.

1. Set $f(u,v) \leftarrow 0$ for all $(u,v) \in E$.
2. While there exists an augmenting path P in G_f:
   (a) Let $\delta = \min_{(u,v)\in P} c_f(u,v)$.
   (b) For each forward edge $(u,v) \in P$, set $f(u,v) \leftarrow f(u,v) + \delta$.
   (c) For each backward edge $(v,u) \in P$, set $f(v,u) \leftarrow f(v,u) - \delta$.
3. Return f.

3.5 Example: The General Network

We now return to the general network from Section 1 and trace the Ford–Fulkerson algorithm (using BFS) from start to finish. Recall the network:



All flows start at zero. The residual graph initially equals the original network.

Iteration 1. BFS from s finds the shortest augmenting path $s \to a \to b \to t$. Bottleneck: $\delta = \min(9, 4, 10) = 4$. Push 4 units.

Edge	Update
$s \to a$	$f : 0 \to 4$
$a \to b$	$f : 0 \to 4$
$b \to t$	$f : 0 \to 4$

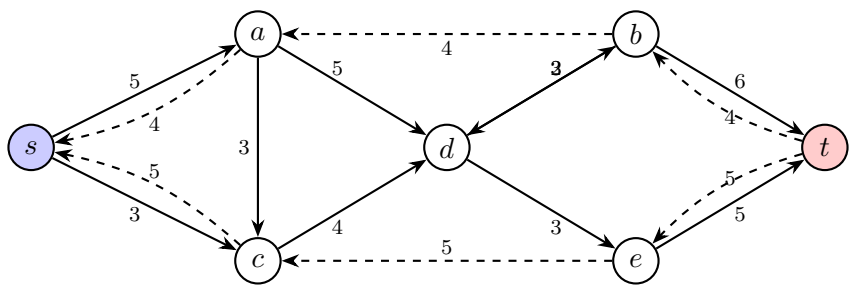
Total $|f| = 4$. The edge $a \to b$ is now saturated.

Iteration 2. BFS finds $s \to c \to e \to t$. Bottleneck: $\delta = \min(8, 5, 10) = 5$. Push 5 units.

Edge	Update
$s \to c$	$f : 0 \to 5$
$c \to e$	$f : 0 \to 5$
$e \to t$	$f : 0 \to 5$

Total $|f| = 9$. The edge $c \to e$ is now saturated.

The residual graph after two iterations:



The two direct short paths ($s \to a \to b \to t$ and $s \to c \to e \to t$) have been partially or fully used. All remaining augmenting paths must route through the central node d.

Iteration 3. BFS finds the length-4 path $s \to a \to d \to b \to t$. Bottleneck: $\delta = \min(5, 5, 3, 6) = 3$. Push 3 units.

Edge	Update
$s \to a$	$f : 4 \to 7$
$a \to d$	$f : 0 \to 3$
$d \to b$	$f : 0 \to 3$
$b \to t$	$f : 4 \to 7$

Total $|f| = 12$. The edge $d \to b$ is now saturated; flow now enters b from two sources (a and d).

Iteration 4. With $d \to b$ saturated, BFS finds $s \to a \to d \to e \to t$. Bottleneck: $\delta = \min(2, 2, 3, 5) = 2$. Push 2 units.

Edge	Update
$s \to a$	$f : 7 \to 9$
$a \to d$	$f : 3 \to 5$
$d \to e$	$f : 0 \to 2$
$e \to t$	$f : 5 \to 7$

Total $|f| = 14$. The edges $s \to a$ and $a \to d$ are now saturated.

Iteration 5. With $s \to a$ saturated, the only forward edge from s is $s \to c$ (residual capacity 3). BFS explores: from c, the edge $c \to e$ is saturated, so only $c \to d$ (capacity 4) is available. From d, both $d \to b$ and $d \to e$ have limited residual capacity.

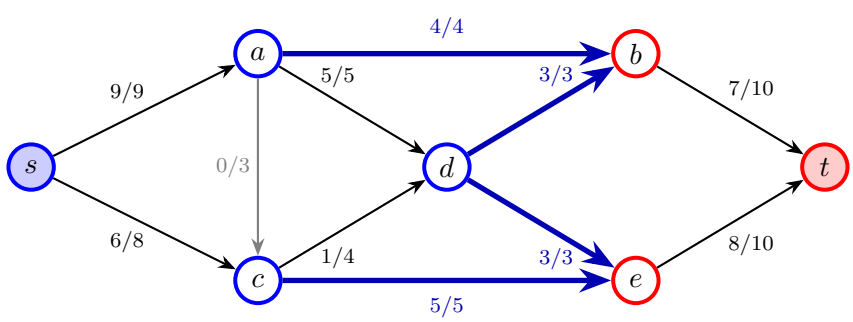
BFS finds $s \to c \to d \to e \to t$. Bottleneck: $\delta = \min(3, 4, 1, 3) = 1$. Push 1 unit.

Edge	Update
$s \to c$	$f : 5 \to 6$
$c \to d$	$f : 0 \to 1$
$d \to e$	$f : 2 \to 3$
$e \to t$	$f : 7 \to 8$

Total $|f| = 15$. The edge $d \to e$ is now saturated.

Termination. BFS from s: the only forward edge is $s \to c$ (residual capacity 2). From c, we reach d (via $c \to d$, capacity 3). From d, both $d \to b$ and $d \to e$ are saturated. The backward edge from d reaches a, but from a all forward edges are saturated. No path to t exists. The algorithm terminates with maximum flow $|f^*| = 15$.

The final flow on every edge:



Verify conservation at each node:

- a: in $= 9$, out $= 4 + 0 + 5 = 9$.
- b: in $= 4 + 3 = 7$, out $= 0 + 7 = 7$.
- c: in $= 6 + 0 = 6$, out $= 1 + 5 = 6$.
- d: in $= 5 + 1 = 6$, out $= 3 + 3 = 6$.
- e: in $= 5 + 3 = 8$, out $= 8$.

The minimum cut. The vertices reachable from s in the final residual graph are $S^* = \{s, a, c, d\}$: from s we reach c (residual capacity 2 on $s \to c$); from c we reach d (via $c \to d$, residual 3); from d we reach a via the backward edge (since $a \to d$ carries flow 5). But from none of these can we reach b, e, or t—every crossing edge is saturated.

The minimum cut is $(\{s, a, c, d\},\ \{b, e, t\})$ with capacity

$$c(a, b) + c(d, b) + c(c, e) + c(d, e) = 4 + 3 + 5 + 3 = 15 = |f^*|.$$

This confirms the max-flow min-cut theorem: the maximum flow equals the capacity of the tightest bottleneck. The bottleneck is not at s (total outgoing capacity 17) or at t (total incoming capacity 20), but at the internal boundary between $\{s, a, c, d\}$ and $\{b, e, t\}$—the four edges that cross from the source side to the sink side are all fully saturated.

4 The Max-Flow Min-Cut Theorem

4.1 Statement

Theorem 3 (Ford–Fulkerson, 1956)**.** *In any flow network* (G, c, s, t)*,*

$$\max_f |f| = \min_{(S,T)} c(S, T).$$

The following are equivalent: (i) f is a maximum flow; (ii) there is no augmenting path in the residual graph G_f; (iii) $|f| = c(S, T)$ for some cut (S, T).

4.2 Proof

In Section 1 we showed that $|f| \le c(S, T)$ for every flow f and every cut (S, T). It remains to show that equality is achieved.

Claim. There exist f^* and (S^*, T^*) with $|f^*| = c(S^*, T^*)$.

Proof. Run Ford–Fulkerson until no augmenting path exists; call the resulting flow f^*. Define

$$S^* = \{v \in V : v \text{ reachable from } s \text{ in } G_{f^*}\}, \qquad T^* = V \setminus S^*.$$

Since $t \notin S^*$, this is a valid cut. For every edge (u, v) with $u \in S^*$, $v \in T^*$ we have $f^*(u, v) = c(u, v)$—otherwise v would be reachable via a forward edge. For every edge (v, u) with $v \in T^*$, $u \in S^*$ we have $f^*(v, u) = 0$—otherwise v would be reachable via a backward edge. Therefore

$$|f^*| = \sum_{u \in S^*,\, v \in T^*} f^*(u, v) - \sum_{v \in T^*,\, u \in S^*} f^*(v, u) = c(S^*, T^*) - 0 = c(S^*, T^*).$$

Since every cut bounds every flow, f^* is a maximum flow and (S^*, T^*) is a minimum cut. □

5 Related Results

The max-flow min-cut theorem is the cornerstone of a family of results in combinatorics.

5.1 Menger's Theorem

Theorem 4 (Menger, 1927)**.** *The maximum number of vertex-disjoint s-t paths equals the minimum number of vertices (other than s, t) whose removal disconnects s from t.*

The edge-disjoint version—max edge-disjoint paths equals min edge cut—is a direct corollary of max-flow min-cut with unit capacities.

5.2 König–Egerváry Theorem

Theorem 5 (König, 1931; Egerváry, 1931)**.** *In a bipartite graph, the size of a maximum matching equals the size of a minimum vertex cover.*

A vertex cover is a set of vertices touching every edge. The minimum vertex cover corresponds to a minimum cut in the flow network.

5.3 Birkhoff–von Neumann Theorem

Theorem 6 (Birkhoff, 1946; von Neumann, 1953)**.** *Every doubly stochastic matrix is a convex combination of permutation matrices.*

A **doubly stochastic matrix** is a square matrix with non-negative entries whose rows each sum to 1 and whose columns each sum to 1. A **permutation matrix** is the special case where every entry is 0 or 1—it picks exactly one entry per row and one per column (i.e. a perfect matching).

The theorem says that every doubly stochastic matrix can be written as a weighted average of permutation matrices. In the language of matchings: if we allow fractional matchings (assigning a weight between 0 and 1 to each edge, so that every vertex's weights sum to at most 1), then the optimal fractional matching has the same value as the optimal integer matching. Relaxing the problem to allow fractions does not help—the integer solution is already optimal.